serviceOrchestration HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestration"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestration` service of the DynamicServiceOrchestration system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation pull . . . 4
3.2 operation subscribe . . . 5
3.3 operation unsubscribe . . . 7

4 Data Models **8**
4.1 struct AuthorizationTokenGenerationResponse . . . 8
4.2 struct ErrorMessage . . . 8
4.3 struct MetadataRequirement . . . 8
4.4 struct OrchestrationNotifyInterface . . . 8
4.5 struct OrchestrationRequest . . . 8
4.6 struct OrchestrationResponse . . . 9
4.7 struct OrchestrationResult . . . 9
4.8 struct OrchestrationServiceRequirement . . . 9
4.9 struct OrchestrationSubscriptionRequest . . . 10
4.10 struct QoSRequirement . . . 10
4.11 struct ServiceInstanceInterfaceResponse . . . 10
4.12 Primitives . . . 11

5 Go and Rust implementation variants **12**
5.1 Go implementation . . . 12
5.2 Rust implementation . . . 12

6 References **12**

7 Revision History **13**
7.1 Amendments . . . 13
7.2 Quality Assurance . . . 13

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `serviceOrchestration` service interface of the DynamicServiceOrchestration system, which provides service orchestration: a consumer asks which provider it should use for a service, and receives the matching, authorized providers with the interfaces to call.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestration` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestration – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestration` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`pull`	POST	`/pull`
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe/{id}`

3 Service Operations

The interfaces of the `serviceOrchestration` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestration` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 POST /serviceorchestration/orchestration/pull

Operation: pull
Input: OrchestrationRequest
Output: OrchestrationResponse

Executes a dynamic orchestration for the requester: service discovery in the ServiceRegistry, removal of locked, blacklisted and unauthorized candidates, interface matching (optionally with translation), preferences, QoS evaluation, matchmaking, exclusive reservation and token generation. Without local results and with `ALLOW_INTERCLOUD`, or directly with `ONLY_INTERCLOUD`, the Gatekeeper is asked (local extension).

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/pull HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "serviceRequirement": {
    "serviceDefinition": "temperatureReading",
    "operations": [
      "FILTER"
    ],
    "versions": [
      "1.0.0"
    ],
    "alivesAt": "2026-10-22T12:00:00Z",
    "metadataRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "interfaceTemplateNames": [
      "string"
    ],
    "interfaceAddressTypes": [
      "string"
    ],
    "interfacePropertyRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "securityPolicies": [
      "string"
    ],
```

```
    "preferredProviders": [
      "string"
    ]
  },
  "orchestrationFlags": {
    "MATCHMAKING": true
  },
  "qualityRequirements": [
    {
      "type": "PING",
      "operation": "FILTER",
      "requirements": {
        "maxLatency": 100
      }
    }
  ],
  "exclusivityDuration": 60
}
```

Listing 1: An pull invocation.

```
{
  "results": [
    {
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "cloudIdentitifer": "LOCAL",
      "providerName": "TemperatureSensor",
      "serviceDefinitition": "temperatureReading",
      "version": "1.0.0",
      "aliveUntil": "string",
      "exclusiveUntil": "string",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ],
  "warnings": [
    "string"
  ]
}
```

Listing 2: An pull response.

3.2 POST /serviceorchestration/orchestration/subscribe?trigger=*{trigger}*

Operation: subscribe
Input: OrchestrationSubscriptionRequest
Output: String

Creates (or replaces) a push subscription of the requester for itself; with `trigger=true` the orchestration is executed and pushed immediately. Returns the subscription identifier, 201 for a new subscription.

Query parameters: *trigger* (Boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /serviceorchestration/orchestration/subscribe?trigger=<trigger> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystemName": "TemperatureDisplay",
  "orchestrationRequest": {
    "serviceRequirement": {
      "serviceDefinition": "temperatureReading",
      "operations": [
        "FILTER"
      ],
      "versions": [
        "1.0.0"
      ],
      "alivesAt": "2026-10-22T12:00:00Z",
      "metadataRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "interfaceTemplateNames": [
        "string"
      ],
      "interfaceAddressTypes": [
        "string"
      ],
      "interfacePropertyRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "securityPolicies": [
        "string"
      ],
      "preferredProviders": [
        "string"
      ]
    },
    "orchestrationFlags": {
      "MATCHMAKING": true
    },
    "qualityRequirements": [
      {
        "type": "PING",
        "operation": "FILTER",
        "requirements": {
          "maxLatency": 100
        }
      }
    ],
    "exclusivityDuration": 60
  },
  "notifyInterface": {
```

```
    "protocol": "HTTPS",
    "properties": {
      "address": "192.168.0.20",
      "port": "8080",
      "method": "POST",
      "path": "/orchestration/notify"
    }
  },
  "duration": 3600
}
```

Listing 3: An subscribe invocation.

```
"string"
```

Listing 4: An subscribe response.

3.3 DELETE /serviceorchestration/orchestration/unsubscribe/*{id}*

Operation: **unsubscribe**
Input: **String**

Removes a subscription of the requester. Returns 200 if removed, 204 if it did not exist.

Path parameter: *id*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceorchestration/orchestration/unsubscribe/<id> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An unsubscribe invocation.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.12, which are used to represent things like hashes, identifiers and texts.

4.1 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

4.2 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.3 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

4.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

4.5 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

4.6 struct OrchestrationResponse

The orchestration result.

Object Field	Value Type	Description
"results"	Array<OrchestrationResult>	The orchestration results, best first.
"warnings"	Array<String>	Warnings of the orchestration, e.g. `autoMatchmaking`, `notExclusive`, `interCloud`.

4.7 struct OrchestrationResult

One provider of the requested service.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"cloudIdentitifer"	String	Cloud identifier of the provider (`LOCAL` for the own cloud). The JSON name keeps the upstream spelling.
"providerName"	String	Name of the provider system.
"serviceDefinitition"	String	Service definition name. The JSON name keeps the upstream spelling.
"version"	Version	Version of the service instance.
"aliveUntil"	String	Time until which the service instance is registered (its expiry), if any.
"exclusiveUntil"	String	Time until which the instance is reserved exclusively for the consumer, if reserved.
"metadata"	Object	Metadata of the service instance.
"interfaces"	Array<ServiceInstanceInterface Response>	Interfaces the consumer can use (only matching ones).
"authorizationTokens"	Object<Object<AuthorizationToken GenerationResponse>>	Authorization tokens generated for the consumer, per interface template and operation.

4.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

4.9 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

4.10 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

4.11 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

4.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8441 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8441". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-dynamic`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/DynamicServiceOrchestration`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	